ESCOLA DE ENGENHARIA DE PIRACICABA
CIÊNCIA DA COMPUTAÇÃO

SERRALHERIA GARCIA: SISTEMA WEB DE GESTÃO DE PEDIDOS E ESTOQUE

Rafael Nunes Villa Rubia	-	RA: 202410057
Diego Ventura Ottani	-	RA: 202410248
João Alex Garcia	-	RA: 202410228
Eduardo de Araujo Quartarolo	-	RA: 202410201
Arthur Santos de Oliveira	-	RA: 202410223

Professor: José Martins Junior

PIRACICABA
2026

1 INTRODUÇÃO

Este relatório descreve à aplicação web desenvolvida para a Serralheria Garcia, empresa de Piracicaba/SP que fabrica portões, grades e estruturas metálicas. O trabalho dá continuidade ao projeto extensionista iniciado nas disciplinas de Banco de Dados I (BDI), na qual foi modelado o banco de dados, e de Programação Orientada à Objetos (POO), na qual foi construída uma aplicação desktop sobre esse banco (ESCOLA DE ENGENHARIA DE PIRACICABA, 2026).

Nesta etapa, a solução foi reescrita como uma aplicação web dividida em duas partes que se comunicam por microsserviços REST: um backend em Java com o framework Spring Boot (BROADCOM, 2026), que concentra as regras de negócio e a persistência por meio de JPA e Hibernate (HIBERNATE, 2026) sobre o SGBD MySQL (ORACLE, 2026), e um frontend em Vue.js (VUE.JS, 2026), que oferece as telas utilizadas pelos funcionários da serralheria.

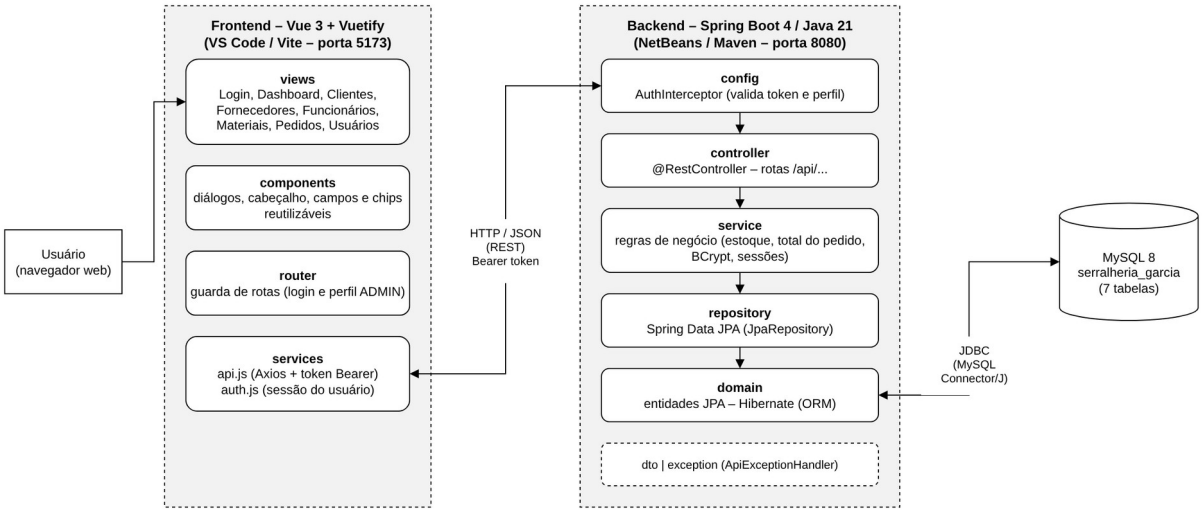
O sistema permite controlar o cadastro de clientes, fornecedores, funcionários e materiais, registrar pedidos (solicitações de serviço) com seus itens, acompanhar o estado de pagamento e manter o estoque de materiais atualizado automaticamente. O acesso é protegido por login e senha, com dois perfis de usuário: Administrador e Funcionário.

O relatório está organizado da seguinte forma: a seção 2 apresenta a arquitetura da aplicação; a seção 3, o banco de dados; a seção 4, as funcionalidades; a seção 5, o layout das telas; a seção 6, os procedimentos de instalação; a seção 7, o roteiro de teste; a seção 8, o atendimento aos requisitos do trabalho; e a seção 9, as considerações finais. Os diagramas foram elaborados na ferramenta draw.io (JGRAPH, 2026).

2 ARQUITETURA DA APLICAÇÃO

A aplicação segue uma arquitetura cliente-servidor em três camadas físicas: o navegador executa o frontend, que se comunica com o backend por requisições HTTP no padrão REST, trocando dados em JSON; o backend, por sua vez, acessa o banco de dados MySQL por meio do Hibernate. A Figura 1 apresenta a visão geral dessa arquitetura.

Figura 1 – Arquitetura geral da aplicação



2.1 Tecnologias utilizadas

O Quadro 1 relaciona as tecnologias empregadas em cada camada da solução.

Quadro 1 – Tecnologias utilizadas

Camada	Tecnologia	Observação
Banco de dados	MySQL 8 (compatível com MariaDB)	Banco `serralheria_garcia`, evoluído a partir do modelo de BDI/POO
Backend	Java 21, Spring Boot 4.1, Spring Web MVC	Projeto Maven aberto e executado no NetBeans (pasta `serralheria-api`)
Persistência	Spring Data JPA e Hibernate	Entidades mapeadas com `@Entity`, `@ManyToOne` e `@OneToMany`
Segurança	BCrypt (spring-security-crypto) e token de sessão	Senhas gravadas com hash; token enviado no cabeçalho `Authorization`
Validação	Jakarta Bean Validation	`@NotBlank`, `@Email`, `@Size`, `@PositiveOrZero` etc.
Frontend	Vue 3, Vue Router, Vuetify 3 e Axios	Projeto Vite editado no VS Code (pasta `serralheria-web`)
Interface	Material Design Icons e CSS próprio	Temas claro e escuro com a identidade visual da serralheria

2.2 Organização do backend

O backend adota uma arquitetura em camadas, separando responsabilidades conforme os padrões estudados na disciplina: MVC, Repository (equivalente ao DAO), DTO e injeção de dependência. O Quadro 2 descreve os pacotes do projeto.

Quadro 2 – Pacotes do backend

Pacote	Responsabilidade
`controller`	Controladores REST (`@RestController`): recebem as requisições HTTP, validam o corpo e delegam aos serviços.
`service`	Regras de negócio: validação de CPF/CNPJ e telefone, cálculo do valor do pedido, baixa e devolução de estoque e controle de sessões.
`repository`	Interfaces Spring Data JPA (`JpaRepository`) responsáveis pelo acesso ao banco.
`domain`	Entidades JPA que representam as tabelas do banco e seus relacionamentos.
`dto`	Objetos de transferência (records) para requisições e respostas.
`exception`	Exceções da aplicação e `ApiExceptionHandler`, que converte erros em respostas JSON padronizadas com o status HTTP adequado (400, 401, 403, 404 e 409).
`config`	Configuração de CORS, interceptador de autenticação, anotação `@SomenteAdmin` e criação do administrador inicial.

2.3 Organização do frontend

O frontend foi construído com Vue 3 e a biblioteca de componentes Vuetify (VUETIFY, 2026). Sua estrutura de pastas é apresentada no Quadro 3.

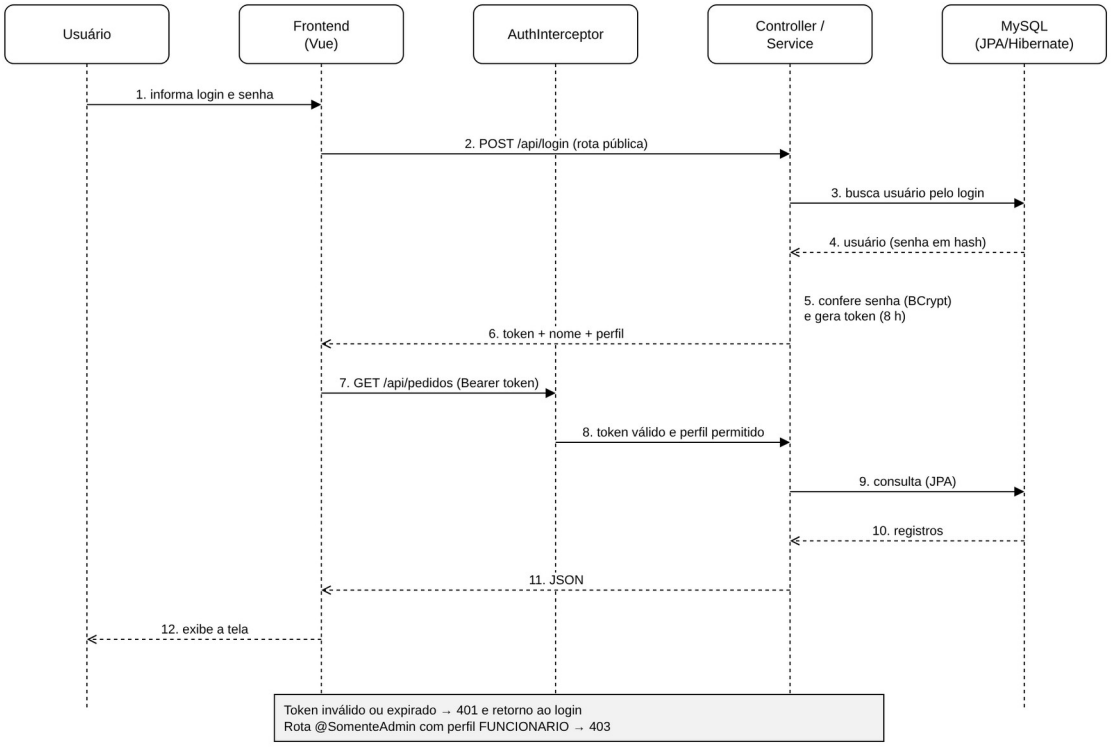
Quadro 3 – Pastas do frontend

Pasta	Conteúdo
`src/views`	Telas: Login, Dashboard, Clientes, Fornecedores, Funcionários, Materiais, Pedidos, Detalhe do Pedido e Usuários.
`src/components`	Componentes reutilizáveis: cabeçalho de página, diálogos de pedido e item, campo de moeda, chip de estado de pagamento, diálogo de confirmação e logotipo.
`src/services`	`api.js` (cliente Axios com token e tratamento de sessão expirada), `auth.js` (sessão do usuário), formatação e notificações.
`src/router`	Rotas e guarda de navegação que impede o acesso sem login e restringe a tela de usuários ao administrador.
`src/plugins`	Configuração do Vuetify: temas claro e escuro nas cores grafite e laranja.

2.4 Comunicação e autenticação

O frontend consome a API REST do backend no endereço `http://localhost:8080/api`, configurável no arquivo `serralheria-web/.env`. Após o login, o backend devolve um token que o frontend armazena e envia em todas as requisições seguintes no cabeçalho `Authorization: Bearer <token>`. O interceptador `AuthInterceptor` valida o token e o perfil do usuário antes de executar qualquer rota, exceto `/api/login`. A Figura 2 ilustra esse fluxo.

Figura 2 – Fluxo de login e de requisição autenticada



2.5 Rotas REST

O Quadro 4 apresenta as principais rotas disponibilizadas pelo backend.

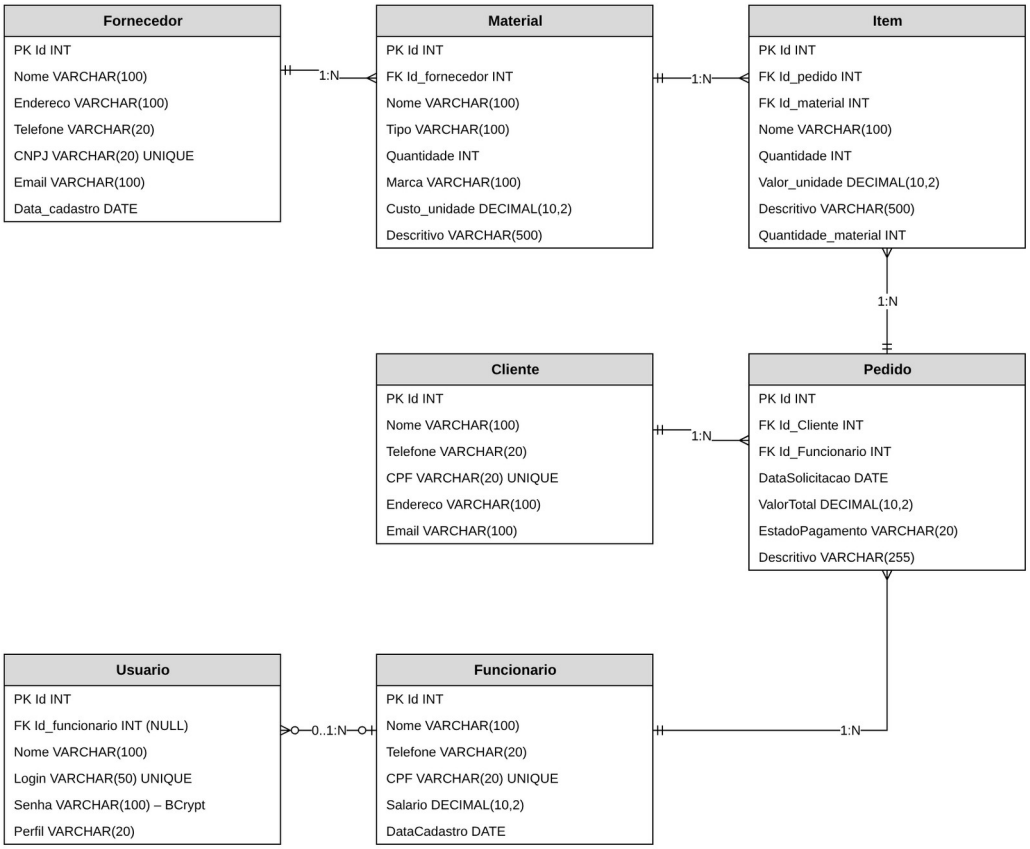
Quadro 4 – Principais rotas REST

Método	Rota	Descrição
POST	`/api/login`	Autentica com login e senha e devolve o token (rota pública)
POST / GET	`/api/logout`, `/api/me`	Encerra a sessão / retorna os dados do usuário logado
GET	`/api/dashboard`	Indicadores da tela inicial
GET, POST, PUT, DELETE	`/api/clientes`	CRUD de clientes (`/{id}` para buscar, alterar e remover)
GET, POST, PUT, DELETE	`/api/fornecedores`	CRUD de fornecedores
GET, POST, PUT, DELETE	`/api/funcionarios`	Consulta para todos; inclusão, alteração e remoção somente para ADMIN
GET, POST, PUT, DELETE	`/api/materiais`	CRUD de materiais (estoque)
GET, POST, PUT, DELETE	`/api/pedidos`	CRUD de pedidos
POST, PUT, DELETE	`/api/pedidos/{id}/itens`	Inclusão, alteração e remoção de itens de um pedido
GET, POST, PUT, DELETE	`/api/usuarios`	Gestão de usuários (somente ADMIN)

3 BANCO DE DADOS

O banco `serralheria_garcia` é o mesmo modelado em BDI e utilizado em POO, com melhorias para a versão web. O script de criação (DDL) está no arquivo `banco/ddl_serralheria.sql`; os dados de demonstração estão em `banco/dados_exemplo.sql`; e o arquivo `banco/serralheria_completo.sql` reúne ambos. A Figura 3 apresenta o diagrama entidade-relacionamento.

Figura 3 – Diagrama entidade-relacionamento do banco serralheria_garcia



3.1 Tabelas

O Quadro 5 descreve as sete tabelas do banco e suas chaves estrangeiras.

Quadro 5 – Tabelas do banco de dados

Tabela	Descrição	Chaves estrangeiras
Cliente	Pessoas físicas (CPF) ou jurídicas (CNPJ) que solicitam serviços	—
Fornecedor	Empresas que fornecem materiais (CNPJ único)	—
Funcionario	Funcionários da serralheria (CPF único, salário)	—
Usuario	Usuários do sistema: login único, senha em hash BCrypt e perfil (ADMIN ou FUNCIONARIO)	Id_funcionario → Funcionario (opcional)
Material	Materiais em estoque: tipo, marca, quantidade e custo unitário	Id_fornecedor → Fornecedor
Pedido	Solicitação de serviço: data, estado de pagamento (Pendente, Pago, Atrasado) e valor total	Id*Cliente* → *Cliente; Id*Funcionario → Funcionario
Item	Itens do pedido: produto/serviço, quantidade, valor unitário e quantidade de material consumida	Id*pedido* → *Pedido; Id*material → Material

3.2 Relacionamentos 1:N

A aplicação acessa sete tabelas relacionadas. Os relacionamentos um-para-muitos implementados e mapeados com JPA são:

a) **Pedido 1:N Item**: mapeado nos dois sentidos (`@OneToMany` com cascata em `Pedido` e `@ManyToOne` em `Item`); é o relacionamento central do sistema, pois o valor do pedido é a soma dos itens;
b) **Cliente 1:N Pedido** e **Funcionario 1:N Pedido**: cada pedido pertence a um cliente e possui um funcionário responsável;
c) **Fornecedor 1:N Material**: cada material é fornecido por um fornecedor;
d) **Material 1:N Item**: cada item de pedido consome um material do estoque;
e) **Funcionario 1:N Usuario**: um usuário do sistema pode estar vinculado a um funcionário.

3.3 Melhorias em relação à versão de POO

Em relação ao banco utilizado na disciplina de POO, foram realizadas as seguintes melhorias:

a) criação da tabela **Usuario** para o controle de acesso com login e senha;
b) renomeação da coluna CPF do fornecedor para **CNPJ**;
c) alteração das quantidades de **Material** e **Item** para o tipo numérico (INT), permitindo o controle de estoque;
d) cálculo automático do **ValorTotal** do pedido pela soma dos itens;
e) inclusão de restrições de unicidade para CPF/CNPJ e login.

No backend, a propriedade `spring.jpa.hibernate.ddl-auto=validate` faz o Hibernate conferir, na inicialização, se as entidades estão de acordo com as tabelas criadas pelo script DDL.

4 FUNCIONALIDADES

Esta seção descreve as funcionalidades oferecidas ao usuário, agrupadas por módulo.

4.1 Controle de acesso

O controle de acesso apresenta as seguintes características:

a) tela de login com usuário e senha; as senhas são armazenadas com hash BCrypt, nunca em texto puro, e não são devolvidas nas respostas da API;
b) cada login gera um token de sessão com validade de 8 horas; ao expirar, o usuário é redirecionado automaticamente para o login com um aviso;
c) dois perfis: ADMIN, com acesso total, incluindo a gestão de usuários e de funcionários, e FUNCIONARIO, com acesso às demais telas e somente consulta de funcionários; a restrição é aplicada no backend (anotação `@SomenteAdmin`, resposta 403) e no frontend (menu e botões ocultos);
d) na primeira execução com a tabela de usuários vazia, o backend cria automaticamente o administrador padrão (`admin` / `admin123`);
e) não é possível remover o próprio usuário nem deixar o sistema sem administrador; ao alterar o perfil de um usuário, suas sessões abertas são encerradas.

4.2 Tela inicial (Dashboard)

Apresenta um resumo da serralheria: total de clientes, total de pedidos, valor já recebido (pedidos pagos) e valor a receber (pendentes e atrasados); barras com a distribuição dos pedidos por estado de pagamento; alerta de materiais com estoque baixo (até 10 unidades); e a lista dos últimos pedidos, além de um atalho para criar um novo pedido.

4.3 Cadastros de clientes, fornecedores e funcionários

Cada cadastro possui uma tela com tabela paginada, busca por texto, botão de inclusão e ações de alterar e remover em cada linha, com diálogo de confirmação. Os formulários abrem em janelas de diálogo e validam os campos obrigatórios. O backend valida e formata automaticamente CPF (000.000.000-00), CNPJ (00.000.000/0000-00) e telefone ((00) 00000-0000) e impede documentos duplicados.

4.4 Materiais (estoque)

Cadastro dos materiais utilizados nos serviços, vinculados a um fornecedor, com tipo, marca, quantidade em estoque, custo unitário e descrição. Materiais com estoque baixo são destacados na tabela. Não é permitido remover um material que já tenha sido utilizado em itens de pedidos.

4.5 Pedidos (solicitação de serviço)

É a principal funcionalidade do sistema. A tela de pedidos lista todas as solicitações, com busca por cliente, funcionário ou número e filtro rápido por estado de pagamento. Ao criar um pedido, informam-se o cliente, o funcionário responsável, o estado de pagamento e a descrição; a data de solicitação é registrada automaticamente.

Na tela de detalhe do pedido são exibidos os dados do cliente, o valor total e a tabela de itens. Cada item informa o produto ou serviço, a quantidade, o valor unitário, o material utilizado e a quantidade de material consumida. O backend aplica as seguintes regras:

a) o valor total do pedido é recalculado automaticamente a cada inclusão, alteração ou remoção de item (soma de quantidade × valor unitário);
b) ao incluir um item, a quantidade de material é baixada do estoque; se o estoque for insuficiente, a operação é recusada com mensagem que informa a quantidade disponível;
c) ao alterar ou remover um item, ou remover o pedido inteiro, o material é devolvido ao estoque;
d) não é possível remover um cliente que possua pedidos vinculados.

4.6 Usuários

Tela exclusiva do administrador para cadastrar, alterar e remover usuários do sistema, definindo login, senha (mínimo de quatro caracteres), perfil e, opcionalmente, o funcionário vinculado.

4.7 Recursos de interface

Atendendo aos requisitos desejáveis do trabalho, a interface oferece:

a) identidade visual própria (logotipo e cores grafite e laranja) e menu lateral agrupado em Cadastros, Solicitação e Administração;
b) alternância entre tema claro e escuro;
c) layout responsivo, com menu recolhível em telas menores;
d) mensagens de sucesso e de erro do backend exibidas em notificações.

5 LAYOUT DAS TELAS

As figuras a seguir apresentam as principais telas da aplicação, utilizando os dados de demonstração do arquivo `serralheria_completo.sql`.

Figura 4 – Tela de login

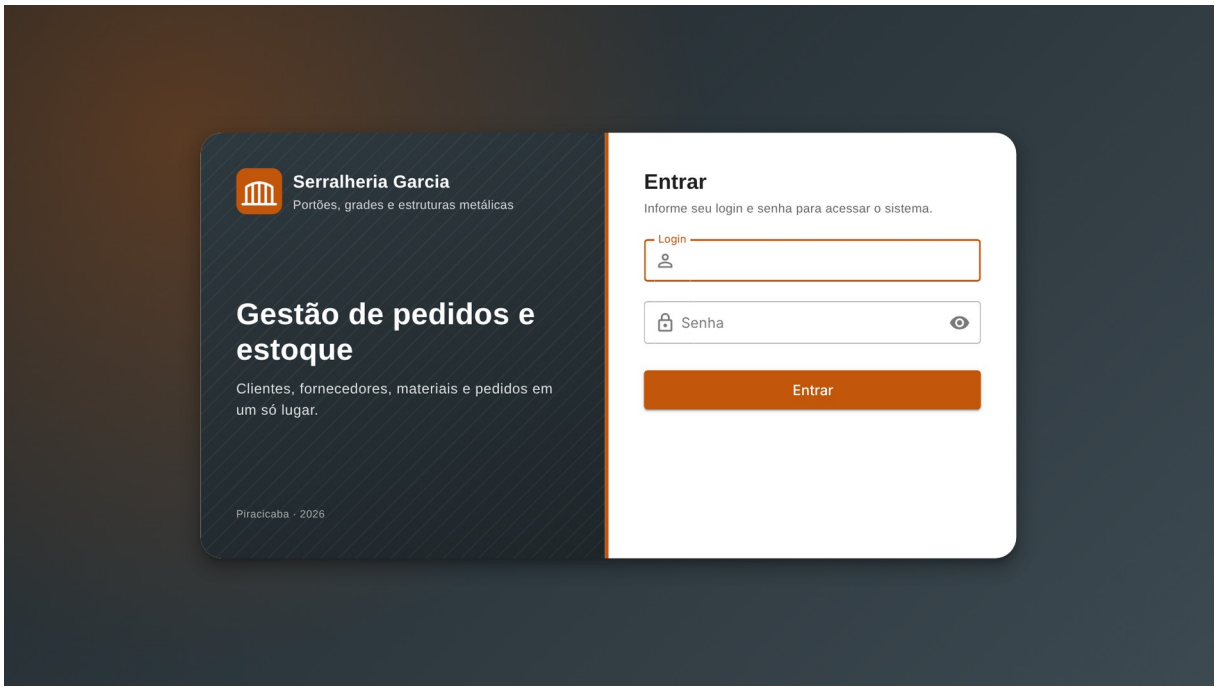


Figura 5 – Tela inicial (Dashboard)

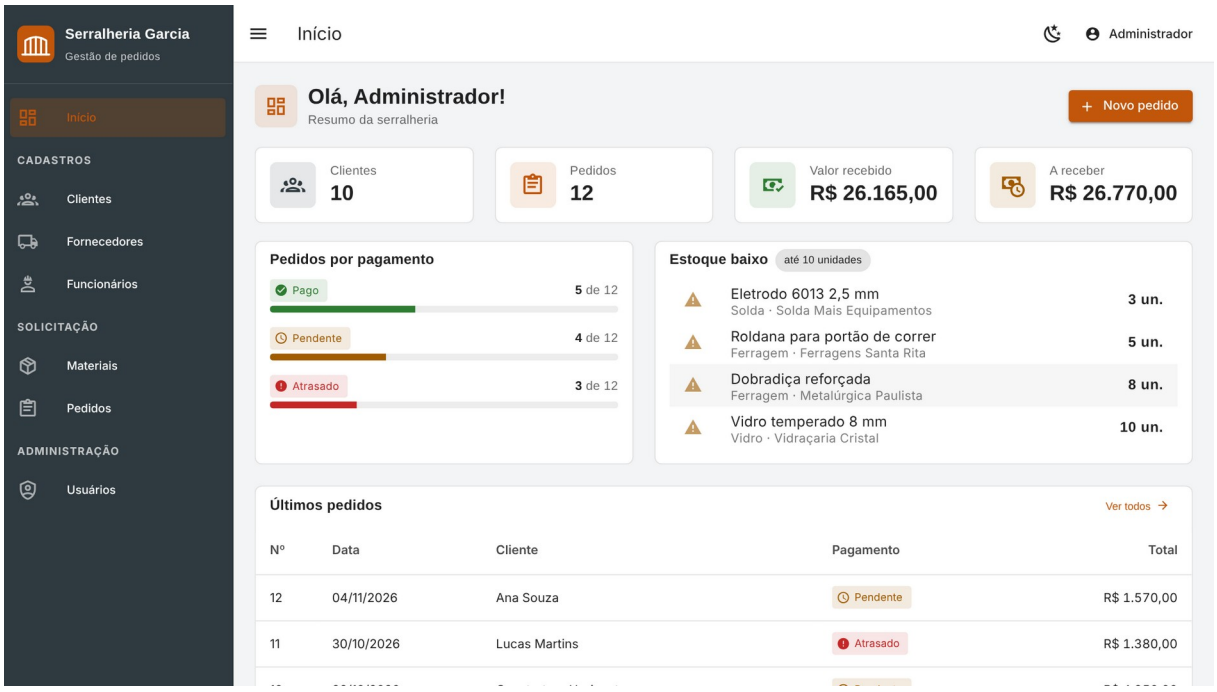


Figura 6 – Cadastro de clientes

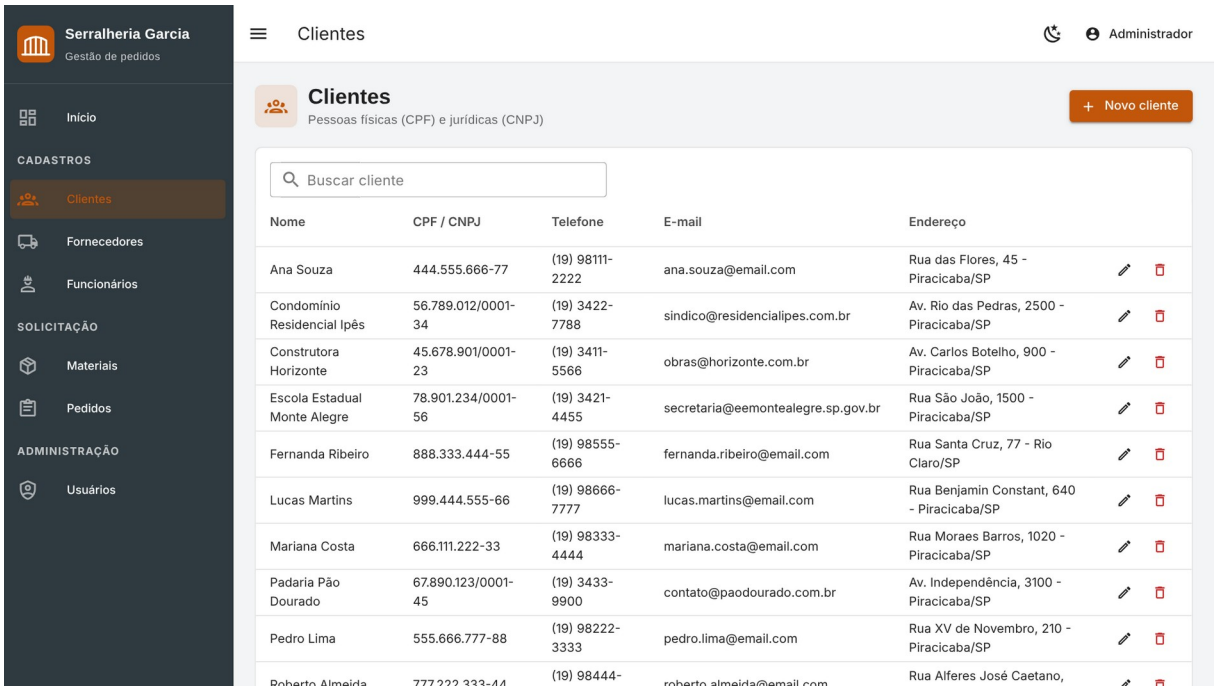


Figura 7 – Formulário de inclusão de cliente

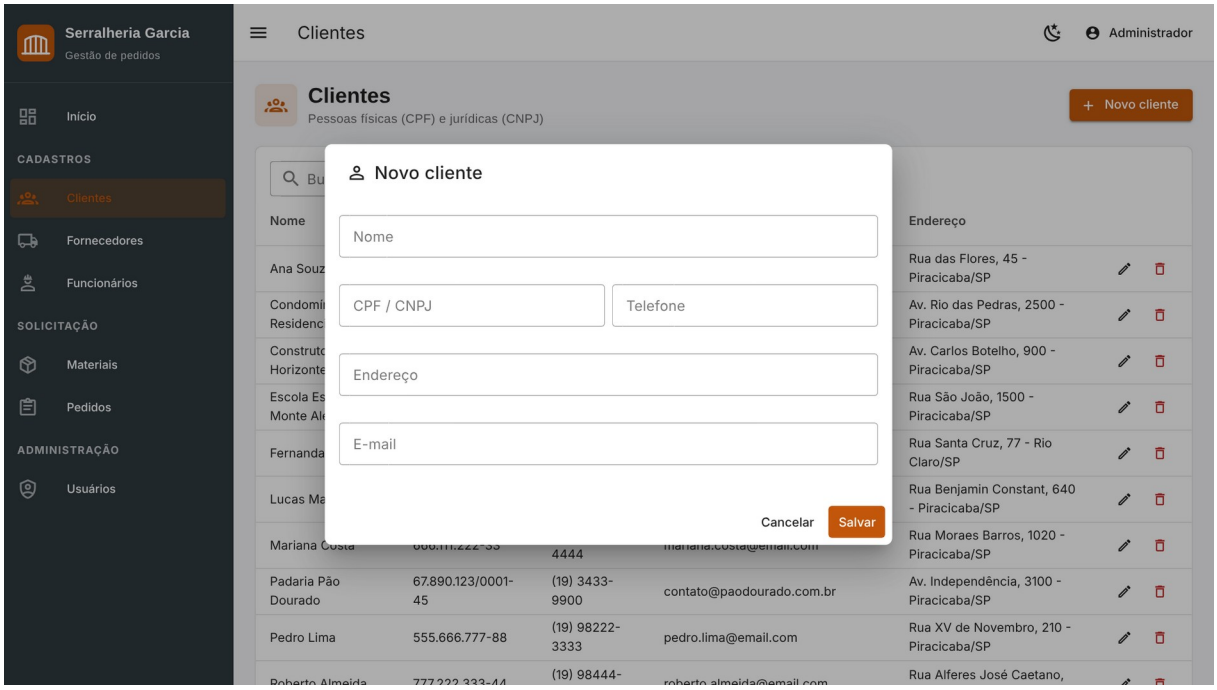


Figura 8 – Cadastro de fornecedores

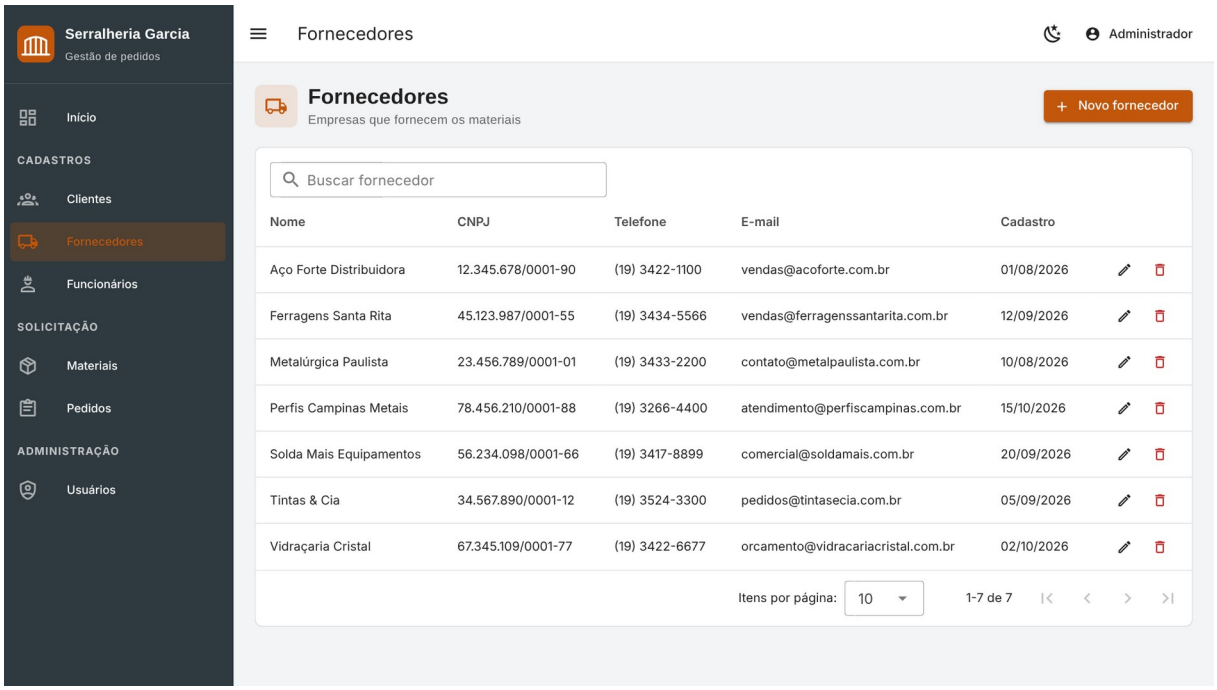


Figura 9 – Cadastro de funcionários

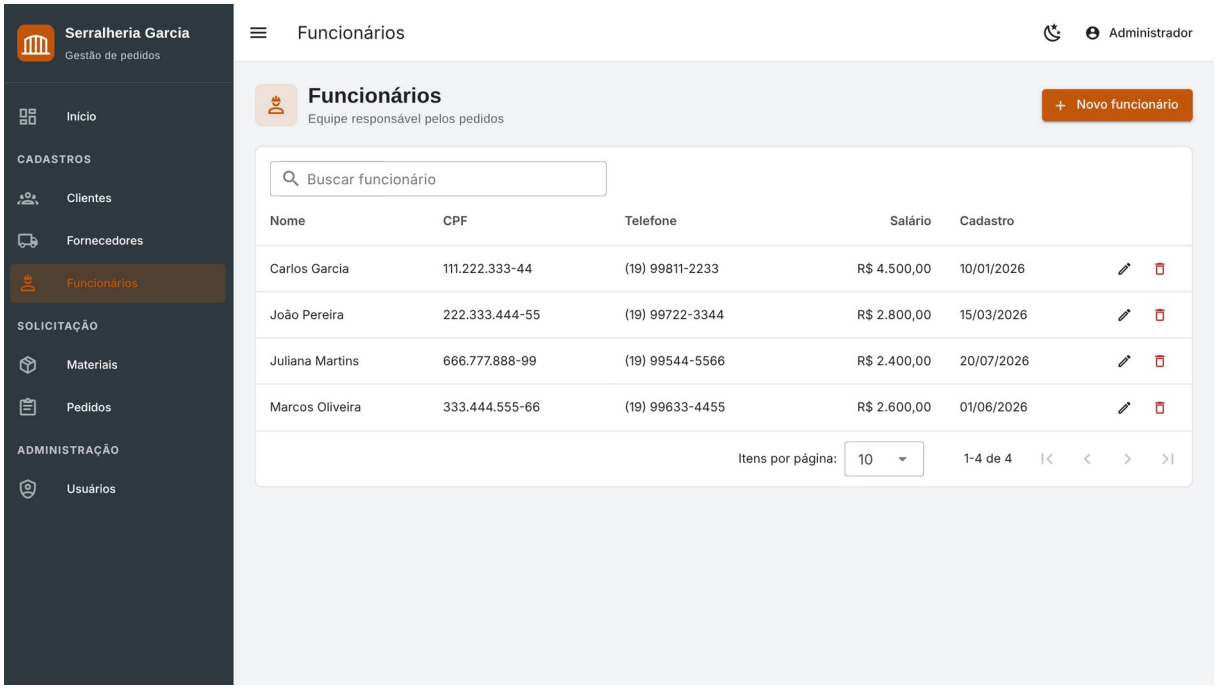


Figura 10 – Cadastro de materiais

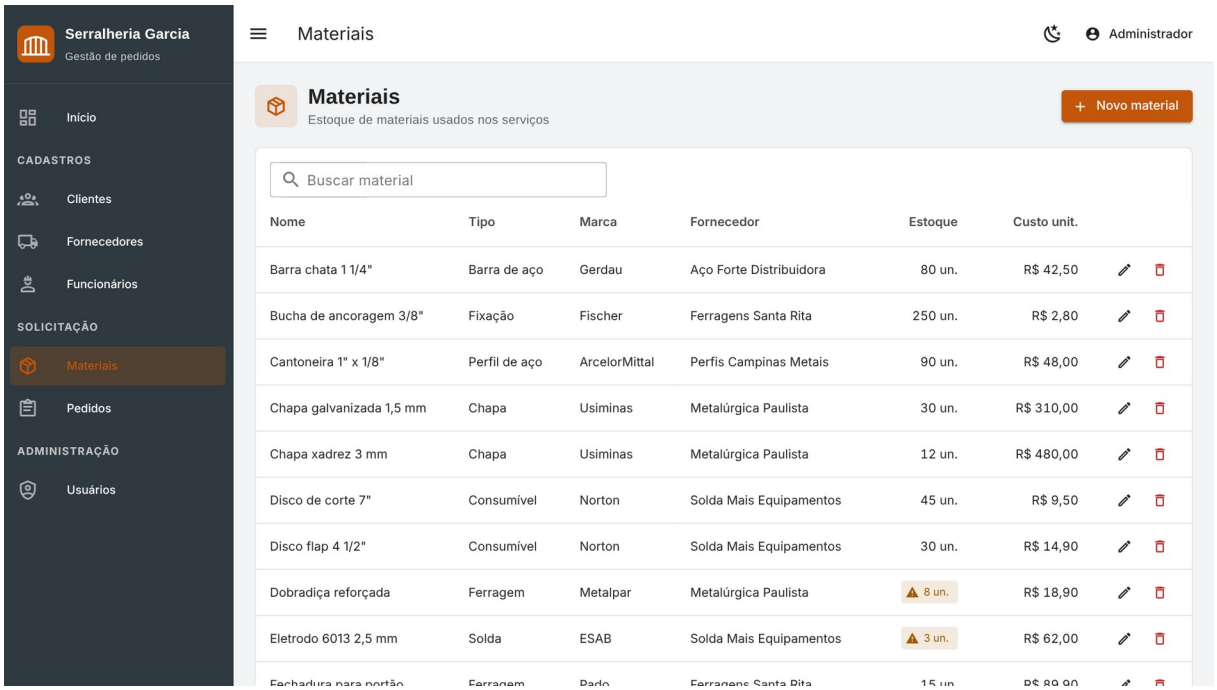


Figura 11 – Formulário de inclusão de material

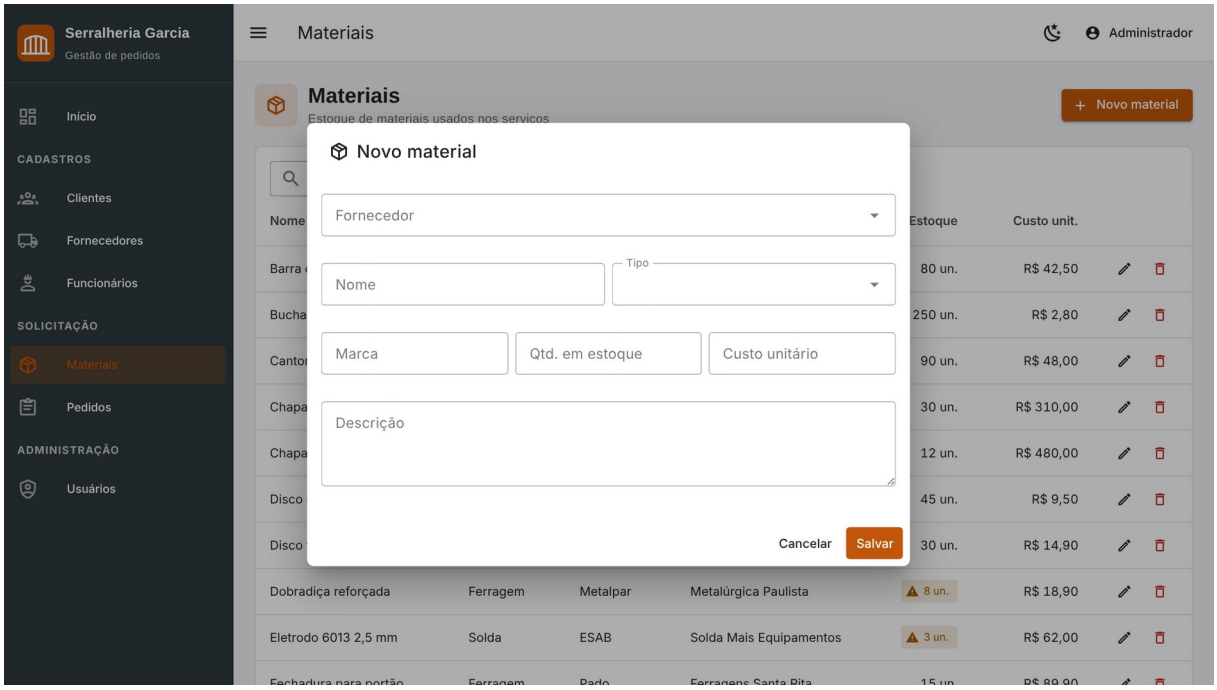


Figura 12 – Lista de pedidos

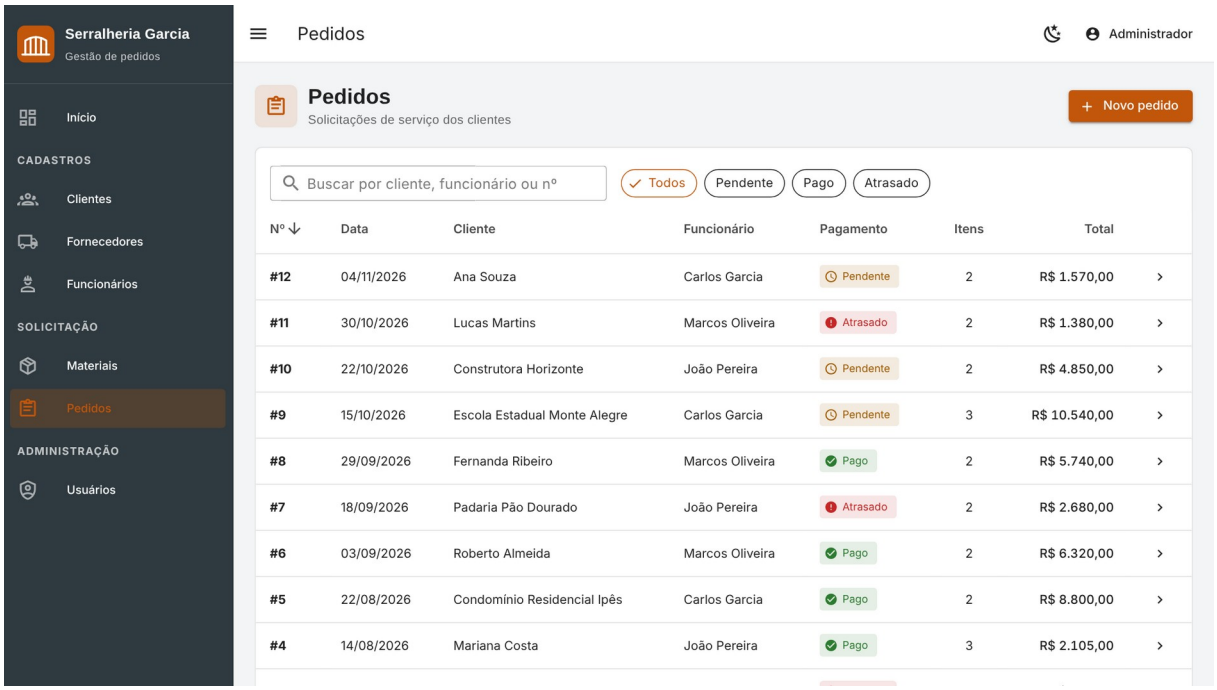


Figura 13 – Formulário de novo pedido

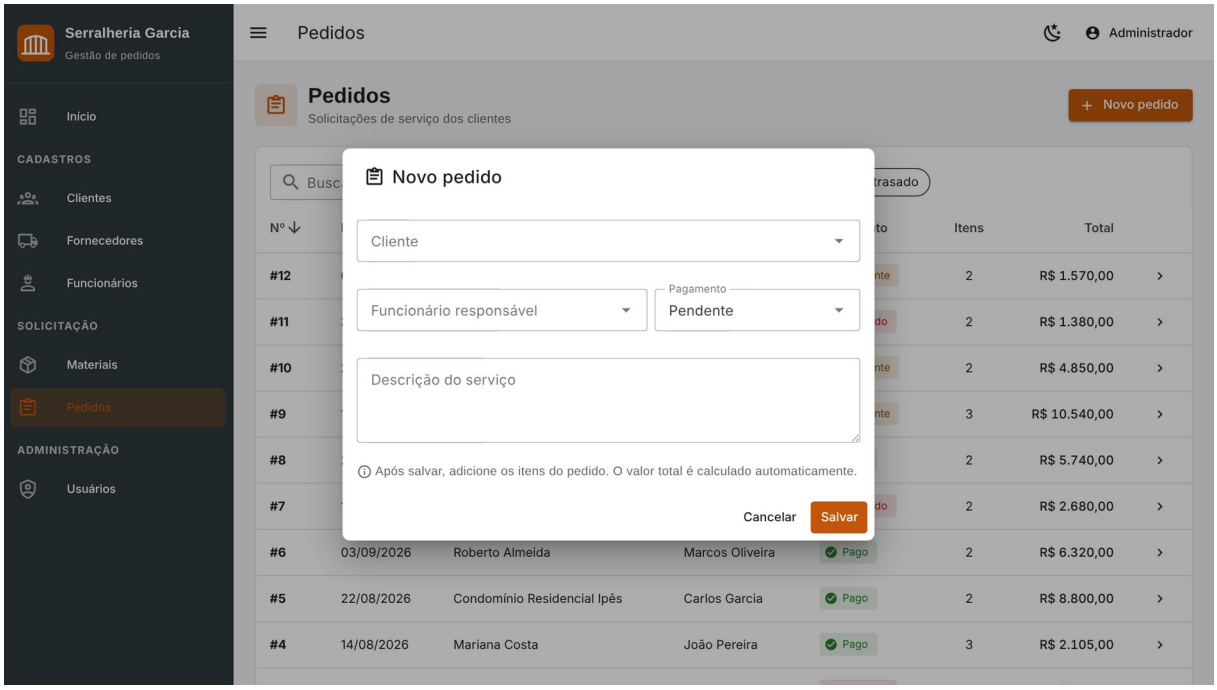


Figura 14 – Detalhe do pedido

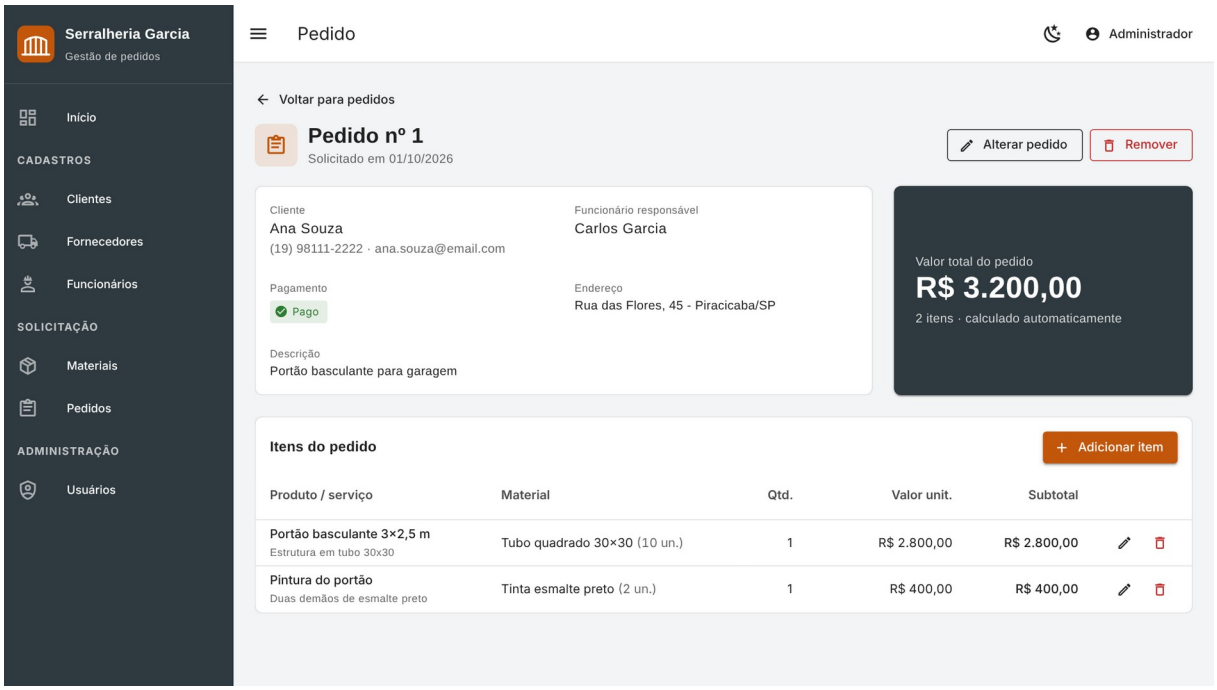


Figura 15 – Formulário de inclusão de item

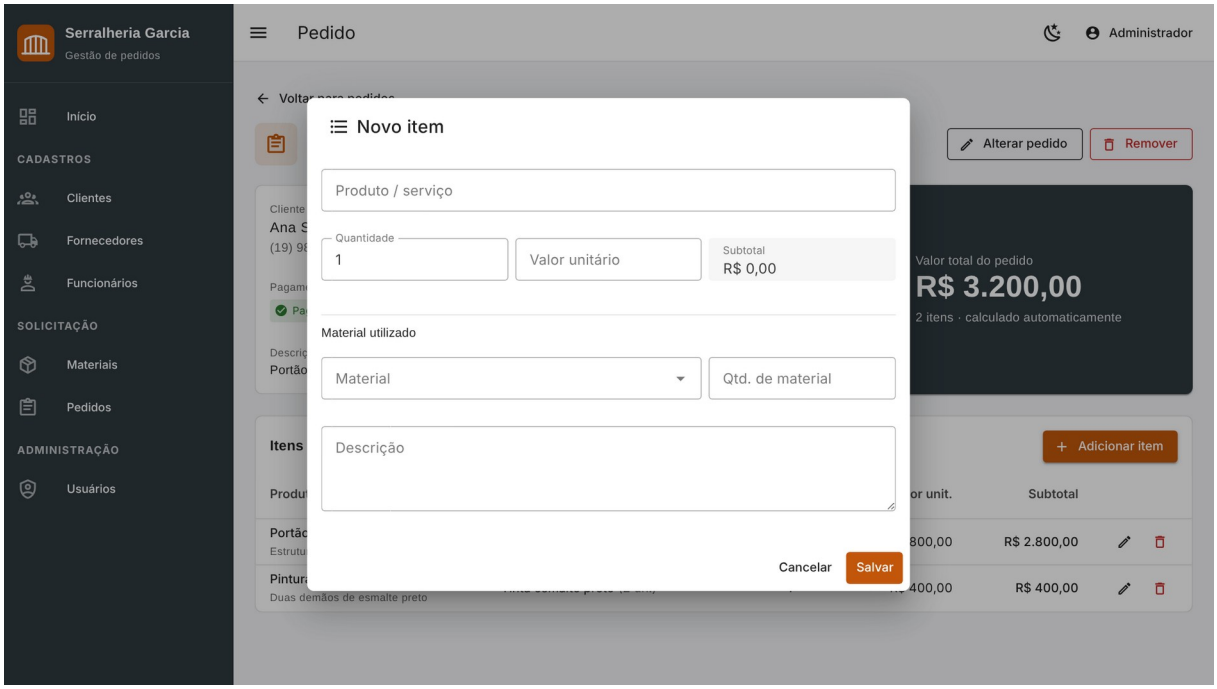
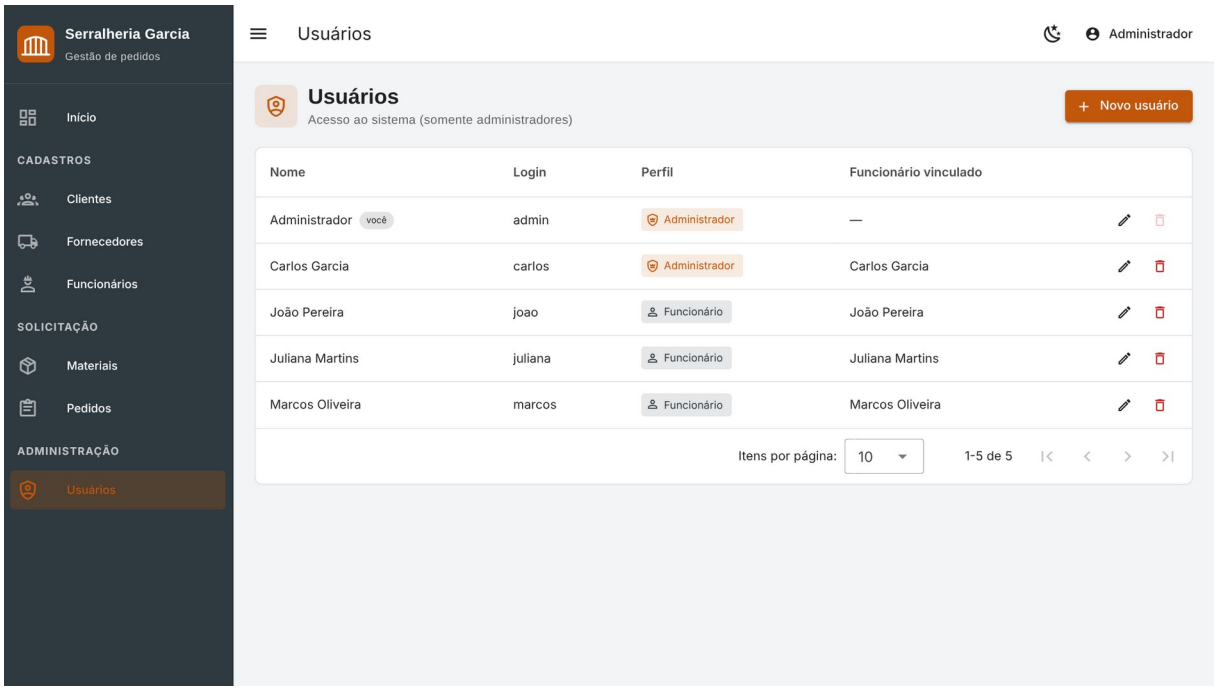


Figura 16 – Gestão de usuários



6 PROCEDIMENTOS DE INSTALAÇÃO E EXECUÇÃO

Esta seção descreve os procedimentos necessários para instalar e executar a aplicação em um computador com Windows ou Linux.

6.1 Pré-requisitos

Antes da instalação, é necessário dispor de:

a) Java JDK 21 ou superior;
b) MySQL Server 8 (ou MariaDB), com o usuário `root` e a senha `1234`, configuração utilizada em `serralheria-api/src/main/resources/application.properties`;
c) Node.js (versão LTS) e npm;
d) Apache NetBeans, para o backend, e Visual Studio Code, para o frontend.

Caso a senha do MySQL seja diferente, basta ajustar a propriedade `spring.datasource.password` no arquivo `application.properties`.

6.2 Criação do banco de dados

Deve-se executar o script completo (estrutura e dados de demonstração) no MySQL. No MySQL Workbench, utiliza-se o menu File → Open SQL Script, seleciona-se o arquivo `banco/serralheria_completo.sql` e executa-se o script. Pela linha de comando do Windows (PowerShell):

```
Get-Content banco\serralheria_completo.sql -Raw -Encoding UTF8 |
  mysql -u root -p1234 --default-character-set=utf8mb4
```

Para criar apenas a estrutura, executa-se somente o arquivo `banco/ddl_serralheria.sql`. Os scripts começam com `DROP DATABASE IF EXISTS` e, portanto, recriam o banco do zero. Após a execução, devem existir as tabelas Cliente, Fornecedor, Funcionario, Item, Material, Pedido e Usuario.

6.3 Execução do backend

No NetBeans, os passos são:

1. acessar File → Open Project e selecionar a pasta `serralheria-api` (projeto Maven);
2. clicar com o botão direito no projeto e escolher Clean and Build (na primeira vez, o Maven baixa as dependências);
3. clicar em Run (F6); a API estará pronta quando o console exibir a mensagem “Started SerralheriaApiApplication”.

Alternativamente, pelo terminal, dentro da pasta `serralheria-api`, executa-se `.\mvnw.cmd spring-boot:run` (Windows) ou `./mvnw spring-boot:run` (Linux). A API fica disponível em `http://localhost:8080/api`.

6.4 Execução do frontend

Abre-se a pasta `serralheria-web` no VS Code e executam-se, no terminal integrado, os comandos:

```
npm install     # somente na primeira vez
npm run dev
```

Em seguida, acessa-se o endereço `http://localhost:5173` no navegador. O endereço da API pode ser alterado no arquivo `serralheria-web/.env` (variável `VITE_API_URL`), por exemplo, para acessar um backend executado em outro computador da rede. No Windows, caso o PowerShell bloqueie a execução do npm, libera-se com o comando `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`.

9 CONSIDERAÇÕES FINAIS

A migração da aplicação desktop para uma arquitetura web com frontend e backend separados permitiu aplicar na prática a divisão em camadas (controller, service, repository e domain), o uso de DTOs, a injeção de dependências do Spring e o mapeamento objeto-relacional com JPA e Hibernate. As regras de negócio ficaram concentradas no backend, o que garante a consistência dos dados (cálculo do valor do pedido, controle de estoque e permissões) independentemente da interface utilizada.

Como evoluções futuras, destacam-se: persistir as sessões em banco ou adotar JWT com Spring Security, já que atualmente os tokens ficam em memória e são perdidos quando o backend é reiniciado; emitir orçamentos em PDF para o cliente; gerar relatórios financeiros por período; e ampliar os testes automatizados das regras de negócio.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BROADCOM. **Spring Boot**. [*S. l.*], 2026. Disponível em: https://spring.io/projects/spring-boot. Acesso em: 9 nov. 2026.

ESCOLA DE ENGENHARIA DE PIRACICABA. **Arquitetura de Software**: definição do trabalho em equipes. Piracicaba: EEP, 2026. Material da disciplina.

HIBERNATE. **Hibernate ORM**. [*S. l.*], 2026. Disponível em: https://hibernate.org/orm/. Acesso em: 9 nov. 2026.

JGRAPH. **draw.io**. [*S. l.*], 2026. Disponível em: https://www.drawio.com/. Acesso em: 9 nov. 2026.

ORACLE. **MySQL 8.0 Reference Manual**. [*S. l.*], 2026. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 9 nov. 2026.

VUE.JS. **Vue.js**: the progressive JavaScript framework. [*S. l.*], 2026. Disponível em: https://vuejs.org/. Acesso em: 9 nov. 2026.

VUETIFY. **Vuetify**: Vue component framework. [*S. l.*], 2026. Disponível em: https://vuetifyjs.com/. Acesso em: 9 nov. 2026.